# EmberDB: A FHIR-Native Time-Series Storage Engine for Continuous Patient Monitoring

Anonymous
Anonymous Institution

***Abstract*—Intensive care unit (ICU) monitors produce heterogeneous vital sign streams that are often represented as HL7 FHIR Observation resources. General-purpose time-series databases such as InfluxDB and TimescaleDB can ingest this data, but they discard clinical semantics at the storage layer (patient identity, LOINC coding, and FHIR resource structure), forcing reconstruction at query time. We present EmberDB, a time-series storage engine that treats FHIR Observations as first-class storage objects. EmberDB partitions data into one-hour time chunks, encodes metrics with a composite key of patient identifier and LOINC code, persists records through a write-ahead log (WAL), and provides built-in clinical pattern detection algorithms: CUSUM changepoint detection, PELT segmentation, seasonal decomposition, Mahalanobis anomaly scoring, and moving-window analysis. We evaluate EmberDB on synthetic MIMIC-IV format chartevents spanning 500 patients over 48-hour ICU stays. EmberDB achieves write throughput exceeding 3.2 million records per second in memory-optimized mode and sustains sub-microsecond point query latency. On clinical workloads, EmberDB outperforms a comparable SQLite baseline on patient-scoped queries and provides native FHIR semantics without post-hoc mapping.**

***Index Terms*—FHIR, time-series database, patient monitoring, ICU, clinical informatics**

## I. Introduction

Modern ICU bedside monitors sample vital signs (heart rate, blood pressure, oxygen saturation, respiratory rate, and temperature) at intervals from one second to five minutes, producing millions of observations per patient-day across a hospital [1]. The HL7 FHIR (Fast Healthcare Interoperability Resources) standard is now a common interoperability format for these observations. Each measurement is encoded as a structured `Observation` resource containing a LOINC-coded type, a numeric value with units, a patient reference, and a timestamp [3].

Storing these observations efficiently while keeping their clinical semantics is an open problem. General-purpose time-series databases such as InfluxDB [4] and TimescaleDB [5] offer high ingestion throughput and good time-range query performance, but they flatten FHIR resources into generic tag/-value pairs, losing the structured relationship between patient, observation code, and clinical context. Relational databases like PostgreSQL preserve schema but lack time-partitioned storage and require expensive joins for temporal queries.

We present **EmberDB**, a Rust time-series storage engine designed for continuous patient monitoring data. EmberDB's contributions are:

1) **FHIR-native storage**: Observations are stored with composite metric keys encoding patient ID, LOINC code, and unit, which removes post-ingestion semantic mapping.
2) **Time-partitioned architecture**: Data is organized into configurable time chunks (default 1 hour) for efficient range scans and natural data lifecycle management.
3) **Built-in clinical pattern detection**: CUSUM changepoint, PELT segmentation, seasonal decomposition, multivariate Mahalanobis anomaly detection, and moving-window analysis are implemented directly in the storage engine.
4) **Write-ahead log persistence**: A WAL with chunk-level durability markers provides crash recovery without sacrificing ingestion performance.

We evaluate EmberDB using synthetic MIMIC-IV format chartevents and compare against a SQLite baseline on identical workloads.

## II. Background and Related Work

### A. Clinical time-series data

ICU monitoring systems produce time-series data with characteristics that differ from typical IoT or infrastructure metrics. Clinical streams are (1) multivariate, with vital signs that are physiologically correlated (e.g., heart rate and blood pressure); (2) semantically rich, with each value requiring a coded type (LOINC), a patient reference, and clinical context; and (3) irregularly sampled, with measurement frequencies that vary by acuity level and clinical protocol.

The MIMIC-IV database [2] documents these properties at scale. Its `chartevents` table contains hundreds of millions of rows across thousands of ICU stays, with item IDs mapping to specific physiological measurements.

### B. Existing storage approaches

**InfluxDB** organizes data by measurement name with tag-based indexing. It works well for infrastructure monitoring, but it requires flattening FHIR Observations into measurement/tag/field triples, which loses the structured patient/code/value relationship.

**TimescaleDB** extends PostgreSQL with hypertables partitioned by time. It preserves relational structure but inherits the overhead of row-based storage and SQL query parsing for high-frequency point lookups.

**SQLite** is a common embedded database for clinical applications. Its simplicity is attractive, but it lacks time-aware partitioning and requires manual index management for temporal queries.

**OpenTSDB** and **QuestDB** offer purpose-built time-series storage but, like InfluxDB, treat all metrics as generic numeric streams without first-class support for healthcare resource models.

## III. System Design

### A. Architecture overview

EmberDB is implemented in Rust and exposes both a library API and a REST interface built on `warp`. The storage engine has three layers: (1) an in-memory chunk manager, (2) a write-ahead log for durability, and (3) a JSON chunk persistence layer.

### B. FHIR-native metric encoding

Each FHIR Observation is converted to an internal `Record` with a composite metric name:

```
metric_name = "{patient_id}|{loinc_code}|{unit}"
```

For example, a heart rate reading for patient P1234 is stored as `P1234|8867-4|/min`. This encoding supports patient-scoped and code-scoped queries without secondary indexes; the storage engine performs prefix matching on the metric key.

Component observations (e.g., blood pressure with systolic and diastolic components) are decomposed into multiple records sharing a common prefix. This keeps the FHIR component structure and allows independent time-series analysis of each component.

### C. Time-partitioned chunks

Records are partitioned into `TimeChunk` structures, each spanning a configurable duration (default: 1 hour). Each chunk maintains:

- A `HashMap<String, Vec<Record>>` mapping metric names to sorted record vectors.
- A resource-type index mapping FHIR resource types to their constituent metric names.
- Chunk metadata including creation time, record count, and dirty flag.

The chunk boundary is computed by flooring the record timestamp to the nearest multiple of the chunk duration, which gives $O(1)$ chunk identification for any timestamp.

### D. Write-ahead log

EmberDB's WAL provides crash recovery by persisting each record before it enters the in-memory chunk. Records are serialized as JSON with a 4-byte length prefix. When a chunk becomes full (exceeding 10,000 records or 1 MB), it is flushed to disk and the corresponding WAL entries are marked durable. On recovery, the engine loads persisted chunks from disk and replays any remaining WAL entries.

### E. MIMIC-IV data loader

EmberDB includes a MIMIC-IV loader module that maps chartevent item IDs to FHIR LOINC codes (e.g., itemid 220045 $\rightarrow$ LOINC 8867-4 for heart rate) and converts rows to `Record` structures. This zero-copy path avoids the overhead of constructing intermediate FHIR JSON representations.

## IV. Pattern Detection

EmberDB provides five built-in pattern detection algorithms, configured via a TOML file and operating directly on stored records.

### A. CUSUM changepoint detection

The Cumulative Sum (CUSUM) algorithm maintains running sums $S^+$ and $S^-$ tracking positive and negative deviations from the series mean:

$$S_i^+ = \max(0,\ S_{i-1}^+ + (x_i - \mu - k))$$

$$S_i^- = \max(0,\ S_{i-1}^- + (\mu - k - x_i))$$

where $k = 0.5\sigma$ is the sensitivity parameter. A changepoint is declared when either sum exceeds the threshold $h = \theta\sigma$, where $\theta$ is configurable (default 2.0).

### B. PELT segmentation

The Pruned Exact Linear Time (PELT) algorithm finds the optimal segmentation by minimizing the penalized cost:

$$\sum_{j=1}^{m+1} \mathcal{C}(y_{\tau_{j-1}+1:\tau_j}) + m \cdot \beta$$

where $\mathcal{C}$ is the Gaussian negative log-likelihood cost for each segment and $\beta$ is the penalty term. Changepoints with magnitude below $\theta\sigma$ are pruned.

### C. Seasonal decomposition

Time series are decomposed into trend $T_t$, seasonal $S_t$, and residual $R_t$ components using moving-average smoothing. Both additive ($Y_t = T_t + S_t + R_t$) and multiplicative ($Y_t = T_t \times S_t \times R_t$) models are supported.

### D. Mahalanobis anomaly detection

For multivariate anomaly detection across correlated vitals, EmberDB computes the Mahalanobis distance:

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Points exceeding a configurable threshold (default 3.0) are flagged as outliers. Metric groups can be specified explicitly or auto-detected via Pearson correlation.

### E. Moving window analysis

A sliding window computes per-window statistics (trend slope, volatility, or range) and flags windows whose statistic deviates from the population by more than $\theta$ standard deviations.

## V. Evaluation

We evaluate EmberDB on three benchmark suites: standalone performance, comparative analysis against SQLite, and a synthetic MIMIC-IV workload. All experiments were run on an Apple Silicon machine running macOS, with EmberDB compiled in release mode (`--release`).

### A. Standalone performance

*1) Write throughput:* Table I shows write throughput across batch sizes with persistence disabled (memory-optimized mode).

TABLE I
EmberDB Write Throughput (Memory Mode)

| Records | Throughput (rec/s) |
|---|---|
| 1,000 | 2,689,372 |
| 10,000 | 1,467,926 |
| 50,000 | 2,582,106 |
| 100,000 | 3,238,495 |
| 500,000 | 3,297,006 |

*2) Query latency:* Table II reports average query latency over 100K ingested records (100 patients, single metric).

TABLE II
EmberDB Query Latency ($\mu$s)

| Query Type | Avg Latency ($\mu$s) |
|---|---|
| Point (narrow range) | 0.1 |
| Latest value | 0.9 |
| Patient + code | 126.2 |
| Full patient (prefix) | 13.6 |
| Time range (50K span) | 61.1 |

*3) Pattern detection:* Table III reports per-invocation latency for each detection algorithm on 2,000 records.

TABLE III
Pattern Detection Latency ($\mu$s)

| Algorithm | Avg Latency ($\mu$s) |
|---|---|
| CUSUM changepoint | 598.0 |
| Seasonal decomposition | 5,167.5 |
| Moving window | 423.5 |
| Mahalanobis (multivariate) | 251.5 |

*4) WAL performance:* With persistence enabled (fsync per record), EmberDB sustains 124 records/s write throughput due to disk synchronization overhead. Recovery from WAL (10,000 records) completes in 0.008 seconds.

*5) Storage overhead:* At 100,000 records with full persistence, EmberDB uses 149.6 bytes per record on disk (14.96 MB total) due to JSON serialization overhead.

### B. Comparative analysis: EmberDB vs. SQLite

*1) Write throughput:* Table IV compares write throughput. EmberDB's in-memory mode avoids disk synchronization overhead, while SQLite uses WAL mode with `PRAGMA synchronous=NORMAL`.

TABLE IV
Write Throughput Comparison (rec/s)

| Records | EmberDB | SQLite | Ratio |
|---|---|---|---|
| 1,000 | 738,257 | 246,038 | 3.0× |
| 10,000 | 1,258,139 | 247,915 | 5.1× |
| 50,000 | 2,145,535 | 282,572 | 7.6× |
| 100,000 | 2,197,044 | 263,857 | 8.3× |
| 500,000 | 2,170,299 | 254,397 | 8.5× |

*2) Query latency:* Table V compares query latency across four query types.

TABLE V
Query Latency Comparison ($\mu$s)

| Query | EmberDB | SQLite | Speedup |
|---|---|---|---|
| Point lookup | 0.1 | 764.5 | 8,890× |
| Latest value | 0.9 | 857.4 | 953× |
| Time range | 63.1 | 808.9 | 12.8× |
| Full patient | 128.2 | 912.6 | 7.1× |

### C. MIMIC-IV synthetic workload

We generated synthetic MIMIC-IV chartevents for 500 patients, each with a 48-hour ICU stay and 6 vital signs sampled every 5 minutes (576 measurements per vital per patient). Vital sign distributions were calibrated to published ICU ranges: HR $\mathcal{N}(84, 17)$, SBP $\mathcal{N}(121, 23)$, DBP $\mathcal{N}(70, 14)$, SpO2 $\mathcal{N}(96.8, 2.8)$, RR $\mathcal{N}(18, 4)$, Temp $\mathcal{N}(98.6, 1.2)$ °F. Each chartevent was mapped to a FHIR Observation via LOINC code and ingested into both EmberDB and SQLite.

*1) Ingestion:* The dataset contains 1,728,000 chartevents (500 patients $\times$ 6 vitals $\times$ 576 measurements). EmberDB ingested the FHIR-converted records at 1,466,955 records/sec (1.18s total). SQLite achieved 160,980 records/sec (10.73s), a 9.1$\times$ speedup for EmberDB.

*2) Query performance:* Table VI reports query latency on the MIMIC workload.

TABLE VI
MIMIC-IV Query Latency ($\mu$s)

| Query | EmberDB | SQLite | Speedup |
|---|---|---|---|
| Single vital (1h) | 3.2 | 30.8 | 9.5× |
| Full patient stay | 944.6 | 616.7 | 0.65× |
| Cohort vital (1h) | 852.4 | 16,773.6 | 19.7× |
| Latest vital | 27.1 | 79.7 | 2.9× |

## VI. Discussion

**FHIR-native advantages.** By encoding patient ID and LOINC code directly into the metric key, EmberDB removes the need for secondary indexes or joins when retrieving patient-scoped observations. This design works well for clinical dashboard queries that retrieve all vitals for a single patient over a shift or stay.

**Pattern detection at the storage layer.** Embedding change-point detection and anomaly scoring directly in the storage engine avoids data export overhead and allows real-time alerting pipelines that operate on the same data path as persistence.

**Limitations.** EmberDB currently serializes chunks as JSON, which has higher storage overhead than columnar formats (149.6 bytes/record vs. $\sim$107 bytes/record for SQLite at 100K records). On the full-patient-stay query in the MIMIC workload, SQLite outperforms EmberDB (617 $\mu$s vs. 945 $\mu$s) because EmberDB requires separate queries per vital metric while SQLite returns all vitals in a single index scan. The single-writer architecture limits concurrent write throughput. The pattern detection algorithms use simplified implementations; production deployment would need validated statistical libraries.

**Future work.** Planned improvements include columnar chunk encoding (e.g., Parquet), concurrent chunk writers with sharded locking, integration with FHIR Subscription for push-based alerts, and support for FHIR Bulk Data export.

## VII. Conclusion

EmberDB shows that a purpose-built time-series engine can provide both high ingestion throughput and native clinical semantics for FHIR Observation data. Encoding patient identity and LOINC codes into the storage key, partitioning data into time-aligned chunks, and integrating pattern detection algorithms gives EmberDB the structured semantics required for continuous patient monitoring without giving up time-series performance. Evaluation on synthetic MIMIC-IV workloads shows competitive performance against SQLite with richer query semantics and no post-hoc data transformation.

## References


[1] A. E. W. Johnson, T. J. Pollard, L. Shen, et al., "MIMIC-III, a freely accessible critical care database," *Scientific Data*, vol. 3, p. 160035, 2016.

[2] A. E. W. Johnson, L. Bulgarelli, L. Shen, et al., "MIMIC-IV, a freely accessible electronic health record dataset," *Scientific Data*, vol. 10, p. 1, 2023.

[3] HL7 International, "FHIR Release 4: Observation Resource," https://hl7.org/fhir/R4/observation.html, 2019.

[4] InfluxData, "InfluxDB: Open Source Time Series Database," https://www.influxdata.com/, 2024.

[5] Timescale Inc., "TimescaleDB: An Open-Source Time-Series SQL Database," https://www.timescale.com/, 2024.